0023. Merge k Sorted Lists

Hard · Linked List

leetcode.com/problems/merge-k-sorted-lists

Problem

- Merge k sorted linked lists into one sorted list.
- Example: [1,4,5],[1,3,4],[2,6] → 1,1,2,3,4,4,5,6.

Intuition

- At each step the next node is the smallest among k current heads; a min-heap gives that in O(log k).
- Alternative: divide and conquer, merging lists in pairs (like merge sort), also O(N log k).

Brute Force

- Merge lists one by one into an accumulator: O(N k).

Optimal Approach

- Push every non-null head into a min-heap keyed by val.
- Pop the smallest, append it, push its next if non-null.
- Repeat until the heap is empty.

Worked Example

```
heap {1a,1b,2c}   pop 1a push 4a
heap {1b,2c,4a}   pop 1b push 3b
heap {2c,3b,4a}   pop 2c push 6c
heap {3b,4a,6c}   pop 3b push 4b
... output 1,1,2,3,4,4,5,6
```

Java Solution

```java
public class MergeKSortedLists {
    public ListNode mergeKLists(ListNode[] lists) {
        PriorityQueue<ListNode> pq = new PriorityQueue<>((a, b) -> Integer.compare(a.val, b.val));
        for (ListNode l : lists) if (l != null) pq.add(l);
        ListNode dummy = new ListNode(0), tail = dummy;
        while (!pq.isEmpty()) {
            ListNode n = pq.poll();
            tail.next = n;
            tail = n;
            if (n.next != null) pq.add(n.next);
        }
        return dummy.next;
    }
}
```

Complexity

- Time: O(N log k), N = total nodes. Space: O(k).

Pitfalls

- Skip null lists when seeding the heap.
- Use Integer.compare in the comparator to avoid subtraction overflow.